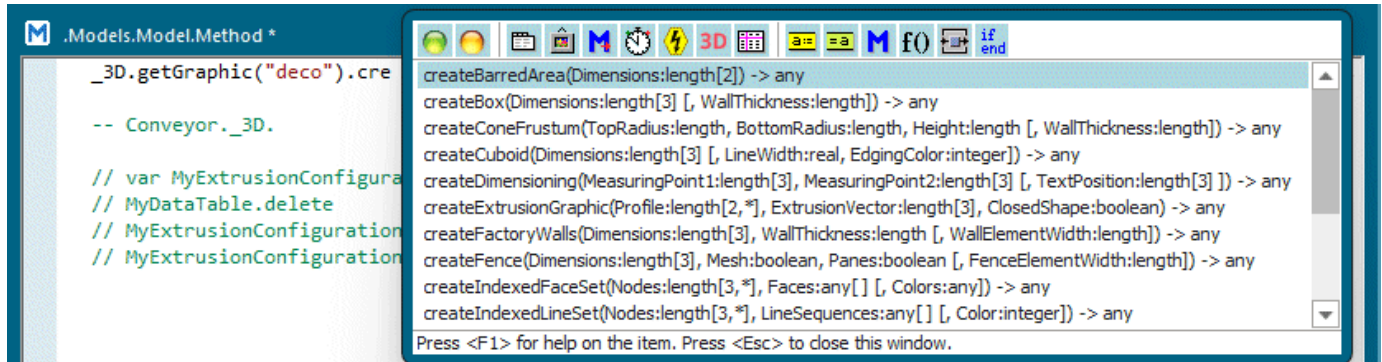


Accessing Graphics

__Accessing Graphics

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for accessing graphics.

You can view and access the **attributes** and **methods** for accessing graphics by clicking **Auto Complete** on the **Edit** ribbon tab of the *Method Editor*.



See also

[Auto Complete](#)

[_3D.calculateMUDimensions \[SimTalk\]](#)

Calculates the dimensions of the *MU* designated by <Path> from its graphic.

Type

Method

Syntax

```
<Path>._3D.calculateMUDimensions([OnlyVisibleGroups:boolean:=false])
```

Parameter

The optional parameter *OnlyVisibleGroups* of data type *boolean* sets if the dimensions of all graphic groups are to be computed (*false*) or only of the visible graphic groups (*true*).

After *Plant Simulation* has executed the method, the **Length**, **Width**, and **Height** of the *MU* match the dimensions of the graphic. *Plant Simulation* uses the zero point of the graphic to calculate the booking point.

Default Value of the Parameter

The default value is *false*.

Example

```
.MUs.Part._3D.calculateMUDimensions(true)
```

See also

[Calculate Dimensions from 3D](#)

[_3D.exchangeGraphic \[SimTalk\]](#)

Exchanges the entire graphic of the selected object designated by <Path> with the graphic you imported from a file.

Type

Method

Syntax

```
<Path>._3D.exchangeGraphic(FilePath:string[,  
UseNewGraphicMeasurement:boolean:=false]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *FilePath* of data type *string* sets the path and the name of the file you want to import.
- The optional parameter *UseNewGraphicMeasurement* of data type *boolean* sets if the new graphic measurements are also to be used for the simulation measurements of the *MU*.

If you specify `true` and the object is a *MU* and if the attribute *_3D.ScaleAutomatically* of the loaded *.s3d* file is set to `false`, this attribute is set to `true` again and the graphic measurements are applied to the simulation. In all other cases the method does not execute anything in addition.

Default Value of the Parameter

The default value is `false`.

Return Value

The return value has the data type *boolean*.

Example

```
MyFrame._3D.exchangeGraphic("C:\Program Files\Siemens\Plant Simulation  
2606\3D\s3D-graphics\Lathe.s3d")
```

See also

[Exchange Graphics](#)

[_3D.ScaleAutomatically \[SimTalk\]](#)

[_3D.ExcludeFromShowContentOfLocation \[SimTalk\]](#)

Hides (`true`) or shows (`false`) the object designated by `<Path>` from the outside representation of the object. Also hides *Connectors* that are connected with hidden objects.

Remarks

After assigning `true`, the outside representation of the selected object is only visible within its location, and **not** in the location of its location. This makes, for example, sense for the *EventController*, for *Methods*, for *DataTables*, etc., which are needed to define and control the simulation but not to illustrate the simulated scene.

Type

Attribute

Syntax

```
<Path>._3D.ExcludeFromShowContentOfLocation:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
Station._3D.ExcludeFromShowContentOfLocation := true
```

See also

[Edit 3D Properties \[dialog\]](#) > [Graphics \[tab\]](#) > [Exclude From "Show content" of the Location](#)

[_3D.exportAsJt \[SimTalk\]](#)

Exports the designated graphic or the designated 3D object of the object designated by <Path> as a .JT file.

Remarks

`_3D.exportAsJt` exports only graphics which are part of the object.

Type

Method

Syntax

```
<Path>._3D.exportAsJt(FilePath:string[, OptimizeForCAD:boolean := false,  
ExcludeInvisibleParts:boolean := false]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *FilePath* of data type *string* sets the path to the target file.
- The optional parameter *OptimizeForCAD* of data type *boolean* sets if the structure of the graphic will be optimized for CAD applications such as *NX* (*true*) or for other applications such as *Plant Simulation* or *Teamcenter Visualization* (*false*).

Default Value of the Parameter

The default value is *false*.

- The optional parameter *ExcludeInvisibleParts* of data type *boolean* sets if all graphics of the object (*false* or not set) are going to be exported or if only those graphics that would be visible if you would open the object in a window (*true*)

Default Value of the Parameter

The default value is *false*.

When you export a graphic as a Jt file and when you activate **Optimize for CAD**, captions and logical object are excluded.

Logical objects are:

- *EventController*
- *FlowControl*
- *Cycle*

- *Exporter*
- *Broker*
- *ShiftCalendar*
- *LockoutZone*
- All *InformationFlow* objects
- All *UserInterface* objects
- All *GA* objects

Return Value

The return value has the data type *boolean*.

true if the file could be exported with graphics.

false if nothing was left to be exported.

Example

```
.MaterialFlow.Frame.exportAsJt("D:\MyS3D\MyFrame.jt")
```

See also

[_3D.exportModelingViewBitmap \[SimTalk\]](#)

[_3D.exportPlanningViewBitmap \[SimTalk\]](#)

[_3D.exportAsS3D \[SimTalk\]](#)

[Import Graphics \[command\]](#)

[Export Graphics](#)

[Export Object](#)

[importGraphics \[SimTalk\]](#)

[_3D.exportAsS3D \[SimTalk\]](#)

Exports the graphic data and the designated animation object of the object designated by <Path> as an .s3D (3D File) file.

Remarks

`_3D.exportAsS3D` exports graphics which are part of the object, attributes that are relevant for the appearance and the animation, as well as the 3D part of embedded objects.

Type

Method

Syntax

```
<Path>._3D.exportAsS3D(FilePath:string) → boolean
```

Parameter

The parameter *FilePath* of data type *string* designates the path to the target file.

Return Value

The return value has the data type *boolean*.

Example

```
.MaterialFlow.Frame._3D.exportAsS3D("D:\MyS3D\MyFrame.s3d")
```

See also

[_3D.exportModelingViewBitmap \[SimTalk\]](#)

[_3D.exportPlanningViewBitmap \[SimTalk\]](#)

[_3D.exportAsJt \[SimTalk\]](#)

[Import Graphics \[command\]](#)

[Export Graphics](#)

[Export Object](#)

[importGraphics \[SimTalk\]](#)

[_3D.exchangeGraphic \[SimTalk\]](#)

[_3D.exportModelingViewBitmap \[SimTalk\]](#)

Exports the 3D modeling view scene of the object designated by <Path> as a .PNG file.

Remarks

The parameters *CameraPosition* and *CameraRotation* correspond to the parameters *CameraPosition* and *CameraPosition* of the function *F3DconfigureView*. If you create *SimTalk* code with *F3DconfigureView*, you can also use it with slight modifications here.

Type

Method

Syntax

```
<Path>._3D.exportModelingViewBitmap(FilePath:string, Width:integer,  
Height:integer[, CameraPosition:length[3], CameraRotationX:real,  
CameraRotationZ:real]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *FilePath* of data type *string* sets the path to the target file.
- The parameter *Width* of data type *integer* sets the width in pixels in the target file.
- The parameter *Height* of data type *integer* sets the height in pixels in the target file.
- The optional parameter *CameraPosition* is an *array* of data type *length* with three values. It sets the camera position in the scene.
- The optional parameter *CameraRotationX* of data type *real* sets the rotation angle of the camera around the x-axis.
- The optional parameter *CameraRotationZ* of data type *real* sets the rotation angle of the camera around the z-axis.

Note

You can either specify all three camera parameters or none of them. If you do not specify them, *Plant Simulation* creates a bitmap of the entire scene.

Return Value

The return value has the data type *boolean*.

true if *Plant Simulation* was able to create the bitmap.

Example

```
.Models.Model._3D.exportModelingViewBitmap("D:\FactoryLibraries\MyFrame",  
100, 100, [2,3,2], 2,2)
```

See also

[_3D.exportAsJt \[SimTalk\]](#)

[_3D.exportAsS3D \[SimTalk\]](#)

[F3DconfigureView \[SimTalk\]](#)

[_3D.exportPlanningViewBitmap \[SimTalk\]](#)

[Generate SimTalk Code and Copy to Clipboard \[camera mark\]](#)

Export Bitmap

[_3D.exportPlanningViewBitmap \[SimTalk\]](#)

Exports the 3D planning view scene of the object designated by <Path> as a .PNG file.

Remarks

The parameters *ViewPosition* and *Zoom* correspond to the parameters *ViewPosition* and *Zoom* of the function *F3DconfigurePlanningView*. If you create *SimTalk* code with *F3DconfigurePlanningView*, you can also use it with slight modifications here.

Type

Method

Syntax

```
<Path>._3D.exportPlanningViewBitmap(FilePath:string, Width:integer,  
Height:integer[, ViewPosition:length[2], Zoom:real]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *FilePath* of data type *string* sets the path to the target file.
- The parameter *Width* of data type *integer* sets the width in pixels in the target file.
- The parameter *Height* of data type *integer* sets the height in pixels in the target file.
- The optional parameter *ViewPosition* is an *array* of data type *length* with two values. It sets the view position in the scene.
- The optional parameter *Zoom* of data type *real* sets the zoom factor in the view.

Note

You can only specify both the view position and the zoom factor together. If you do not specify them, *Plant Simulation* creates a bitmap of the entire scene.

Return Value

The return value has the data type *boolean*.

true if *Plant Simulation* was able to create the bitmap.

Example

```
.Models.Model._3D.exportPlanningViewBitmap("D:\FactoryLibraries\MyFrame",  
100, 100, [2,3], 3)
```

See also

[Generate SimTalk Code and Copy to Clipboard \[camera mark\]](#)

[_3D.exportAsJt \[SimTalk\]](#)

[_3D.exportAsS3D \[SimTalk\]](#)

[F3DconfigurePlanningView \[SimTalk\]](#)

[_3D.exportModelingViewBitmap \[SimTalk\]](#)

[Use Planning View](#)

[Export Bitmap](#)

[_3D.getGraphic \[SimTalk\]](#)

Returns the designated graphic of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.getGraphic(GraphicGroupName:string[, Path:any[]]) → void
```

Parameters

You can specify the following parameters:

- The parameter *GraphicGroupName* of data type *string* designates the name of the graphic group.
- The optional parameter *Path* of data type *any* either lists numerical indexes or names. You cannot mix these two types of addressing.

The numerical indexes are one-based.

The names are just that, namely names, for example *MyName*.

Compare these examples:

- The expression `[ ]` designates the entire graphic group. If you do not specify the parameter, you also access the entire graphic group. If you enter `_3D.getGraphic("default", [ ])`, you address the graphic group named **"default"** as a whole.
- The expression `[1]` designates the first direct child graphic.
- The expression `["abc"]` designates the direct child graphic with the name **abc**.
- The expression `[1, 4]` designates the fourth child graphic of the first direct child graphic.
- The expression `["abc", "def"]` designates the child graphic with the name **def** of the direct child graphic with the name **abc**.

The figure below illustrates the values of the parameters *GraphicGroupName* and *Path* for the displayed nodes of the *Frame*.

	Value of the parameter GraphicGroupName	Value of the parameter Path
 Station		
▼  default	"default"	[]
 [1] workplace (▲ P=349 M)	"default"	[1] or ["workplace"]
 [2] rack (▲ P=315 M)	"default"	[2] or ["rack"]
 [3] pane (▲ P=12 M)	"default"	[3] or ["pane"]
 [4] table (▲ P=1137 M)	"default"	[4] or ["table"]
 [5] legs (▲ P=360 M)	"default"	[5] or ["legs"]
▼  [6] safety-switch	"default"	[6] or ["safety-switch"]
 [1] (▲ P=226 M)	"default"	[6, 1]
 [2] (▲ P=121 M)	"default"	[6, 2]
▼  [7] control	"default"	[7]
 [1] (▲ P=31 M)	"default"	[7, 1]
 [2] (▲ P=15 M)	"default"	[7, 2]
 [3] (▲ P=103 M)	"default"	[7, 3]
 [4] (▲ P=103 M)	"default"	[7, 4]
 [5] (▲ P=103 M)	"default"	[7, 5]
 [6] (▲ P=103 M)	"default"	[7, 6]
▼  [8] tool	"default"	[8] or ["tool"]
 [1] (▲ P=66 M)	"default"	[8, 1]
 [2] (▲ P=12 M)	"default"	[8, 2]
 [3] (▲ P=219 M)	"default"	[8, 3]
 [4] (▲ P=289 M)	"default"	[8, 4]
▼  alternative1	"alternative1"	[]
 [1] (■ P=12)	"alternative1"	[1]
▼  [4]	"alternative1"	[2]
 [1] (■ P=284)	"alternative1"	[2, 1]
 [2] (■ P=1568)	"alternative1"	[2, 2]

You can view and access the attributes and methods of the graphics and the graphic groups by selecting **Edit > Auto Complete** in the *Method Editor*.

Return Value

void if the specified graphic does not exist. This way you can query if a graphic exists or not.

Examples

```
/var g := Machine._3D.getGraphic("default", [1])  
// returns the first graphic in the graphic group named default  
g.Position := [1,2,3]
```

```
var g := Machine._3D.getGraphic("default", ["head"])  
// returns the graphic named head in the graphic group named default  
g.Position := [1,2,3]
```

Abbreviated Notation if GraphicGroupName is a Valid Identifier

If the parameter *GraphicGroupName* is a valid identifier, you can also type:

```
var g := Machine._3D.default([1])  
// returns the first graphic in the graphic group named default  
g.Position := [1,2,3]
```

```
var g := Machine._3D.default(["head"])  
// returns the graphic named head in the graphic group named default  
g.Position := [1,2,3]
```

See also

Auto Complete

[_3D.InheritGraphics \[SimTalk\]](#)

[_3D.getObject \[SimTalk\]](#)

[graphic \[SimTalk\]](#)

[_Detailed Access To Graphics](#)

[Copy Object Settings to a Method](#)

[_3D.InheritGraphics \[SimTalk\]](#)

Activates (`true`) or deactivates (`false`) graphic inheritance of the object designated by `<Path>`.

Type

Attribute

Syntax

```
<Path>._3D.InheritGraphics:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.Models.Model.Station1._3D.InheritGraphics := false
var a: boolean := .Models.Model.Station1._3D.InheritGraphics
// returns whether the graphic is inherited or not
```

See also

[Edit 3D Properties \[dialog\]](#) > [Graphics \[tab\]](#)

[_3D.getGraphic \[SimTalk\]](#)

_3D.optimizeObject [SimTalk]

Optimizes the 3D object designated by <Path>.

Remarks

The object can either be a simulation object or an animatable object.

Note

As you cannot restore discarded data, we advise to optimize the structure of the object after you are done modeling the object. Optimizing the 3D graphic improves performance.

We also recommend to save the model under a different name before optimizing it. This way you can return to the state the model had before optimizing it in case the optimized model reports errors or does not meet your expectations.

Type

Method

Syntax

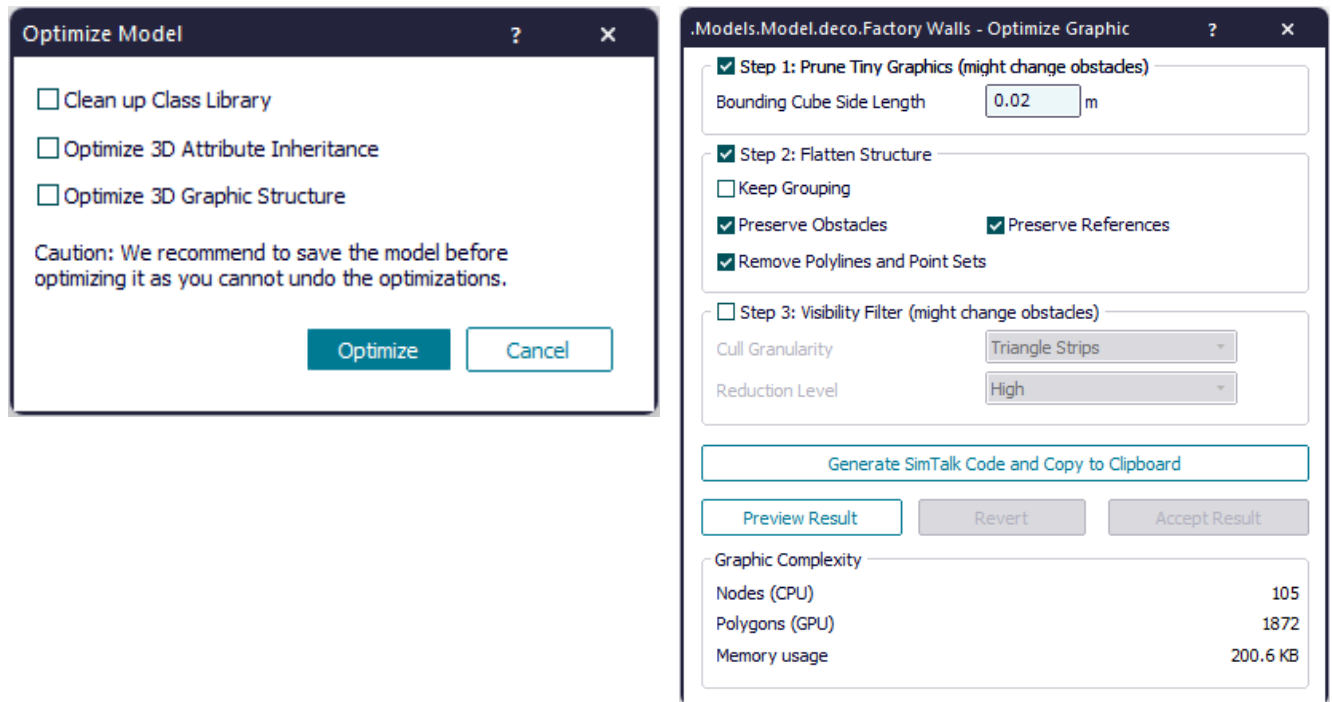
```
<Path>._3D.optimizeObject(WithContainedObjects:boolean,  
OptimizeInheritance:boolean, OptimizeGraphicStructure:boolean)
```

Parameters

You can specify the following parameters:

- The parameter *WithContainedObjects* of data type *boolean* sets if, starting from the object designated by <Path>, the entire partial structure will be optimized, for example the entire model for `basis._3D.optimizeObject` or the entire folder named **MaterialFlow** for the instruction `.MaterialFlow._3D.optimizeObject(true)`. Specify `false` to only optimize the object itself.
- The parameter *OptimizeInheritance* of data type *boolean* sets if setting **Optimize 3D Attribute Inheritance** will be activated and if *Plant Simulation* attempts to (`true`):
 - Activate inheritance of 3D attributes whose values uniquely match the corresponding value of its origin. Graphics inheritance is never activated.
 - **Replace MU Animation Paths** for individual matrix places, with names such as **#0#0**, etc., **with an Animation Area** if this change is possible without a position change of the animation.
- The parameter *OptimizeGraphicStructure* of data type *boolean* sets if the setting **Optimize 3D Graphic Structure** will be activated and if *Plant Simulation* applies the optimization setting **Flatten structure**, without the setting **Keep Grouping** to all editable graphics (`true`).

If this parameter is `true`, *Plant Simulation* optimizes the model with activated setting **Preserve References**. This is intended to prevent that the model size increases after optimizing the model. In some cases it might prevent an optimal optimization result for the graphic structure.



Example

```
.Models.Model._3D.optimizeObject(true, true, false)
```

See also

[Step 2: Flatten Structure](#)

[Preserve References](#)

[Optimize Model](#)

[Optimize Selected Graphic](#)

[Optimizing a Graphic](#)

[Optimizing the Model](#)

[_3D.ShowContent \[SimTalk\]](#)

Shows (`true`) or hides (`false`) the outside representation of the objects that are contained within the object designated by `<Path>` in 3D windows that show the designated object from the outside.

Remarks

The outside representation of an object consists of its visible external graphic groups and, for *Frames* only, the outside representation of its contained objects for which `_3D.ExcludeFromShowContentOfLocation` is set to `false`.

Contained objects, for which `3D.ExcludeFromShowContentOfLocation` is set to `true`, never are visible outward, independent of the setting in the outside representation of this object.

`_3D.ShowContent` only applies for objects, which can contain *MUs* and for the *Frame*. Objects, which can contain *MUs*, are most of the material flow objects, plus *Container* and *Transporter* and their animatable objects.

Type

Attribute

Syntax

```
<Path>._3D.ShowContent : boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
Station._3D.ShowContent := true
```

See also

[Edit 3D Properties \[dialog\]](#) > [Graphics \[tab\]](#) > [Show Content](#)

[_3D.ExcludeFromShowContentOfLocation \[SimTalk\]](#)

Detailed Access To Graphics

Detailed Access To Graphics

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for accessing the object graphics.

addGraphicTransformation [SimTalk]

Adds an additional transformation to an existing transformation of the designated graphic of the object designated by <Path>.

Remarks

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

Let's suppose that a graphic is translated by the coordinates (3, 0, 0) from its origin and its basic orientation, and rotated by 30° around its y-axis.

Applying the method *addGraphicTransformation* to this graphic with a translation of (2, 0, 0) and a uniform scaling factor of 2 results in a graphic translation which first translates it by the coordinates (3, 0, 0), then rotates it by 30° around the y-axis, then translates it by (2, 0, 0), and finally scales the resulting translation by a factor of 2 in all directions. Note the sequence is important. It is, for example, not possible to combine the translations into one.

Type

Method

Syntax

```
<Path>.addGraphicTransformation(Position:length[3], RotationAngle:real,  
RotationAxis:real[3], Scale:real/real[3])
```

Parameters

You can specify the following parameters:

- The parameter *Position* is an *array* of data type *length* with three values. They designate the position of the graphic.
- The parameter *RotationAngle* of data type *real* sets its rotation angle.
- The parameter *RotationAxis* is an *array* of data type *real* with three values. They designate the rotation axis of the graphic.
- The parameter *Scale* sets the scale of the graphic.

You can either specify a single parameter of data type *real*, which sets a uniform scaling.

Or you can specify an *array* of data type *real* with three values. They designate the scaling factors along the X-axis, the Y-axis, and the Z-axis.

Examples

```
MyStation.getGraphic("default", [1,4]).addGraphicTransformation([3,2,1],  
30, [0,0,-1], 2)
```

```
MyStation.getGraphic("deco", [1,4]).addGraphicTransformation([3,2,1], 30,  
[0,0,-1], 2)
```

See also

[Transformation](#)

[_3D.getGraphic \[SimTalk\]](#)

deleteGraphic [SimTalk]

Deletes the specified graphic of the object designated by <Path>.

Remarks

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

Type

Method

Syntax

```
<Path>.deleteGraphic
```

Example

```
MyStation._3D.getGraphic("default", [2,2]).deleteGraphic
```

See also

[_3D.InheritGraphics \[SimTalk\]](#)

graphic [SimTalk]

Returns the designated inner graphic or graphic group of graphic or graphic group designated by <Path>.

Remarks

The method *graphic* is available for composite graphics, compare **InternalGraphicType > Composite Graphics**.

Type

Method

Syntax

```
<Path>.graphic(Name:string)
<Path>.graphic(Index:string)
```

Parameter

The parameter *Name* of data type *string* designates the name of the inner graphic.

Or

The parameter *Index* of data type *integer* designates the n-th inner graphic.

Example

```
var outerGraphic := machine._3D.getGraphic("default", [1])
// returns the first graphic in the graphic group named 'default'
var innerGraphic := outerGraphic.Graphic(2)
// returns the second graphic in the first graphic in the graphic group
// named 'default'
innerGraphic.Position := [1,2,3]
```

See also

[InternalGraphicType \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

[_Detailed Access To Graphics](#)

groupGraphics [SimTalk]

Transforms the designated graphics in the graphic designated by <Path> into a group.

Remarks

Note

groupGraphics is available for composite graphics, compare **InternalGraphicType > Composite Graphics**.

You can specify the graphics to be grouped either by their numeric indexes or by their names.

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

Type

Method

Syntax

```
<Path>.groupGraphics(Children:any[]) → any
```

Parameters

The parameter *Children* is an *array* of data type *any*. It sets the numeric indexes or the names of the graphics which are to be added to the grouped graphic.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Examples

```
MyFrame._3D.getGraphic("default", [1,2]).groupGraphics([2,3])  
// groups the graphics at the indexes [1,2,2] and [1,2,3]
```

```
MyFrame._3D.getGraphic("deco").groupGraphics(["Plate", "Tool"])  
// groups the graphics named "Plate" and "Tool"
```

See also

[InternalGraphicType \[SimTalk\]](#)

Group Graphics

[ungroupGraphic \[SimTalk\]](#)

[_3D.InheritGraphics \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

Index [SimTalk] - graphic in 3D

Returns the numeric index of the specified graphic of the object designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>.Index → integer
```

Return Value

The return value has the data type *integer*.

Examples

```
var index : integer := MyStation._3D.getGraphic("default", ["Wheels"]).Index  
// the Variable contains the numeric index of the graphic
```

```
var index : integer := Graphicgroup.Graphic("Wheels").Index  
// the Variable contains the numeric index of the graphic
```

```
var Graphicgroup := MyStation._3D.getGraphic("default")
```

See also

[Name \[SimTalk\] - graphic](#)

InternalGraphicType [SimTalk]

Returns the type of the graphic or graphic group designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>.InternalGraphicType → string
```

Return Value

The return value has the data type *string*.

Plant Simulation returns one of the following types:

Composite graphics

- **FactoryWalls** are **Factory Walls**. After optimizing a parent graphic this graphic might possibly not be recognized as factory walls but as a Shape or a GroupNode.
- **Fence** is a **Fence**. After optimizing a parent graphic this graphic might possibly not be recognized as a fence but as a GroupNode.
- **GraphicGroup** is a graphic group. This is the case for all graphics whose access arose with an empty array in the second parameter of **_3D.getGraphic** or if you only specified a single parameter.
- **GroupNode** is a normal group node that is formed by **groupGraphics** for example.
- **Instance** is a special group node which can only contain a single child graphic.
- **Rack** is a **Rack**. After optimizing a parent graphic this graphic might possibly not be recognized as a rack but as a Shape or a GroupNode.
- **Stairs** is a **Staircase**. After optimizing a parent graphic this graphic might possibly not be recognized as a set of stairs but as a Shape or a GroupNode.
- **Switch** is a special group node that always shows one if its children only.

Simple graphics

- **BarredArea** is a **Barred Area**. After optimizing a parent graphic this graphic might possibly not be recognized as a barred area but as a Shape.
- **Box** is a **Box**. After optimizing a parent graphic this graphic might possibly not be recognized as a box but as a Shape.

- **Dimensionings** are **Measurements** within your model. After optimizing a parent graphic this graphic might possibly not be recognized as dimensionings but as a Shape.
- **Shape** is a graphic that is not subdivided any more, which contains its graphics data itself, as opposed to GroupNode.

Example

```
var graphic := MyStation._3D.getGraphic("default", [1])
if graphic.InternalGraphicType = "Fence"
    graphic.restoreCutouts
end
```

See also

[_3D.getGraphic \[SimTalk\]](#)

makeAnimatableObject [SimTalk]

Creates an animatable object for the object designated by <Path>.

Remarks

makeAnimatableObject takes the designated graphic, extracts it from the original object, and uses this graphic for the animatable object to be created.

If the graphic, from which the animatable object is created, is not part of the **default** graphics group, the method creates a graphics group with the name of the graphics group where the graphic to be extracted is located. In previous versions *Plant Simulation* created the graphics group within the **default** graphics group.

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

Type

Method

Syntax

```
<Path>.makeAnimatableObject(ObjectName:string,  
ExtractAtBoundingBoxBase:boolean[, ExtractionOffset:length[3],  
SwapXAndZAxis:boolean]) → any
```

Parameters

You can specify the following parameters:

- The parameter *ObjectName* of data type *string* sets the name of the animatable object.
- The parameter *ExtractAtBoundingBoxBase* of data type *real* sets if the insertion position of the new object is determined based on the bounding box of the graphic (*true*) or not (*false*).
- The optional parameter *ExtractionOffset* is an *array* of data type *length* with three values which sets the offset of the insertion position. If you do not specify the value, the method uses the values [0,0,0].
- The optional parameter *SwapXAndZAxis* of data type *boolean* sets if the graphics of the animatable object will be rotated by 90° around the y-axis, thus swapping the x-axis and the z-axis, while looking the same from the outside (*true*). If you do not specify the value, the method uses the value *false*.

Return Value

The return value of data type *any* designates the newly created animatable object.

Examples

```
var tool := MyStation._3D.getGraphic("default",  
["tool"]).makeAnimatableObject("toolHead",true, [0,0,0])  
tool.JointType := "Revolute Joint"
```

```
MyStation._3D.getGraphic("default",  
["tool"]).makeAnimatableObject("toolHead",true, [0,0,0])  
MyStation._3D.getObject("toolHead").JointType := "Revolute Joint"  
// This line of code accomplishes the same as the second line of code in  
the first example.
```

See also

[Make Animatable Object](#)

[delete \[SimTalk\] - animatable object](#)

[_3D.getObject \[SimTalk\]](#)

[_3D.addObject \[SimTalk\]](#)

MaterialActive [SimTalk]

Activates (`true`) or deactivates (`false`) the material of the graphic designated by `<Path>`.

Remarks

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.MaterialActive:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation._3D.getGraphic("default", [3,3]).MaterialActive := true
```

See also

[Set the Material of a Graphic](#)

[MaterialAmbientColor \[SimTalk\]](#)

[MaterialDiffuseColor \[SimTalk\]](#)

[MaterialEmissiveColor \[SimTalk\]](#)

[MaterialShininess \[SimTalk\]](#)

[MaterialSpecularColor \[SimTalk\]](#)

[MaterialTransparency \[SimTalk\]](#)

MaterialAmbientColor [SimTalk]

Sets the ambient color of the material of the specified graphic designated by <Path>.

Remarks

When assigning a value to the attribute, we automatically set the material to active (`MaterialActive := true`).

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.MaterialAmbientColor:integer
```

Assignment Value

You can assign a value of data type *integer*.

You will usually set the color with the method [makeRGBValue \[SimTalk\]](#).

Example

```
MyStation._3D.getGraphic("default", [3,3]).MaterialAmbientColor :=  
makeRGBValue(0,0,255)
```

See also

[Edit 3D Properties \[dialog\]](#) > [Material \[tab\]](#) > [Ambient Color \[material\]](#)

[MaterialActive \[SimTalk\]](#)

[MaterialDiffuseColor \[SimTalk\]](#)

[MaterialEmissiveColor \[SimTalk\]](#)

[MaterialShininess \[SimTalk\]](#)

MaterialSpecularColor [SimTalk]

MaterialTransparency [SimTalk]

MaterialDiffuseColor [SimTalk]

Sets the diffuse color of the graphic designated by <Path>.

Remarks

When assigning a value to the attribute, we automatically set the material to active (`MaterialActive := true`).

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.MaterialDiffuseColor:integer
```

Assignment Value

You can assign a value of data type *integer*.

You will usually set the color with the method [makeRGBValue \[SimTalk\]](#).

Example

```
MyStation._3D.getGraphic("default", [3,3]).MaterialDiffuseColor :=  
makeRGBValue(255,0,0)
```

See also

[Edit 3D Properties \[dialog\]](#) > [Material \[tab\]](#) > [Diffuse Color \[material\]](#)

[MaterialActive \[SimTalk\]](#)

[MaterialAmbientColor \[SimTalk\]](#)

[MaterialEmissiveColor \[SimTalk\]](#)

[MaterialShininess \[SimTalk\]](#)

MaterialSpecularColor [SimTalk]

MaterialTransparency [SimTalk]

MaterialEmissiveColor [SimTalk]

Sets the emissive color of the graphic designated by <Path>.

Remarks

When assigning a value to the attribute, we automatically set the material to active (`MaterialActive := true`).

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.MaterialEmissiveColor:integer
```

Assignment Value

You can assign a value of data type *integer*.

You will usually set the color with the method [makeRGBValue \[SimTalk\]](#).

Example

```
MyStation._3D.getGraphic("default", [3,3]).MaterialEmissiveColor :=  
makeRGBValue(0,255,0)
```

See also

[Edit 3D Properties \[dialog\]](#) > [Material \[tab\]](#) > [Emissive Color \[material\]](#)

[MaterialActive \[SimTalk\]](#)

[MaterialAmbientColor \[SimTalk\]](#)

[MaterialDiffuseColor \[SimTalk\]](#)

[MaterialShininess \[SimTalk\]](#)

MaterialSpecularColor [SimTalk]

MaterialTransparency [SimTalk]

MaterialShininess [SimTalk]

Sets the material shininess of the graphic designated by <Path>.

Remarks

When assigning a value to the attribute, we automatically set the material to active (`MaterialActive := true`).

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.MaterialShininess:real
```

Assignment Value

You can assign a value of data type *real* between 0 and 1.

Example

```
MyStation._3D.getGraphic("default", [3,3]).MaterialShininess := 0.9
```

See also

[Edit 3D Properties \[dialog\]](#) > [Material \[tab\]](#) > [Shininess \[material\]](#)

[MaterialActive \[SimTalk\]](#)

[MaterialAmbientColor \[SimTalk\]](#)

[MaterialDiffuseColor \[SimTalk\]](#)

[MaterialSpecularColor \[SimTalk\]](#)

[MaterialTransparency \[SimTalk\]](#)

MaterialSpecularColor [SimTalk]

Sets the specular color of the graphic designated by <Path>.

Remarks

When assigning a value to the attribute, we automatically set the material to active (`MaterialActive := true`).

You can only access the attribute if the material is active.

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.MaterialSpecularColor:integer
```

Assignment Value

You can assign a value of data type *integer*.

You will usually set the color with the method [makeRGBValue \[SimTalk\]](#).

Example

```
MyStation._3D.getGraphic("default", [3,3]).MaterialSpecularColor :=  
makeRGBValue(0,255,0)
```

See also

[Edit 3D Properties \[dialog\]](#) > [Material \[tab\]](#) > [Specular Color \[material\]](#)

[MaterialActive \[SimTalk\]](#)

[MaterialAmbientColor \[SimTalk\]](#)

[MaterialDiffuseColor \[SimTalk\]](#)

MaterialSpecularColor [SimTalk]

MaterialShininess [SimTalk]

MaterialTransparency [SimTalk]

MaterialTransparency [SimTalk]

Sets the transparency of the material of the graphic designated by <Path>.

Remarks

When assigning a value to the attribute, we automatically set the material to active (`MaterialActive := true`).

You can only access the attribute if the material is active.

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Note

If your model contains several partially or fully transparent graphics behind each other from your current view point, it can happen that some or all of these graphics look wrong or seem to be missing.

This is due to heuristics without which the rendering performance would drastically decrease.

Type

Attribute

Syntax

```
<Path>.MaterialTransparency:real
```

Assignment Value

You can assign a value of data type *real* between 0 and 1.

Example

```
MyStation._3D.getGraphic("default", [3,3]).MaterialTransparency := 0.9
```

See also

[Edit 3D Properties \[dialog\]](#) > [Material \[tab\]](#) > [Transparency \[material\]](#)

[MaterialActive \[SimTalk\]](#)

MaterialTransparency [SimTalk]

MaterialAmbientColor [SimTalk]

MaterialDiffuseColor [SimTalk]

MaterialSpecularColor [SimTalk]

MaterialShininess [SimTalk]

Name [SimTalk] - graphic

Sets the name of the graphic or graphic group designated by <Path>.

Remarks

Assigning the attribute deactivates graphic inheritance.

You cannot change the name of the graphic group named **default**.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.Name:string
```

Assignment Value

You can assign a value of data type *string*.

Examples

```
Station._3D.getGraphic("default", [1]).Name := "MyGraphic"
```

```
var n:string := Station._3D.getGraphic("default", [1]).Name
```

See also

[_3D.InheritGraphics \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

[Index \[SimTalk\] - graphic in 3D](#)

NumGraphics [SimTalk]

Returns the number of the direct child graphics index of the graphic or graphic group designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>.NumGraphics → integer
```

Return Value

The return value has the data type *integer*.

Example

```
var count : integer := MyStation._3D.getGraphic("default",  
[1,2,4]).NumGraphics  
var count : integer := MyStation._3D.getGraphic("deco", [1,2,4]).NumGraphics
```

See also

[_3D.getGraphic \[SimTalk\]](#)

ObstacleForWorker [SimTalk]

Sets the settings for the obstacle for the *Worker* of the graphic designated by <Path>.

Remarks

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Note

The attribute is not available for those objects, which you cannot insert directly into a *Frame*, for example *MUs* or *folders*. Graphics of type **BarredArea** do not provide the attribute either.

Type

Attribute

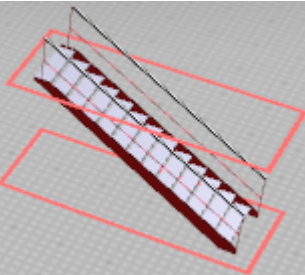
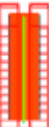
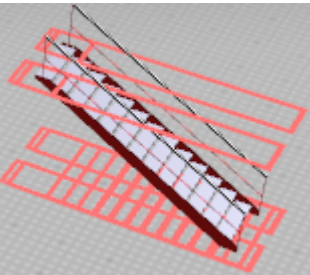
Syntax

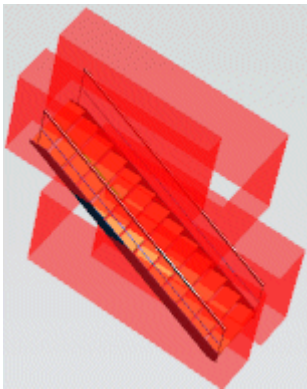
```
<Path>.ObstacleForWorker:string
```

Assignment Value

You can assign a value of data type *string*.

You can enter one of the following settings for graphics of type **Stairs**:

Bounding Box	Graphics	Sides
 	 	

Bounding Box	Graphics	Sides
		

- "(None)"
The staircase is no obstacle for the *Worker* who can walk through it.
- "Bounding box"
The bounding box of the staircase is an obstacle for the *Worker* around which he has to walk.
- "Sides"
This setting computes the obstacle for the *Worker* in a way that:
 - An accessible entrance toward which the staircase lead or an exit from which the staircase leads away is not obscured by the obstacle.
 - The obstacle is limited to the applicable range for the respective simulation-relevant height.

You can specify one of these settings for the graphics types **FactoryWalls**, **Fence**, **GroupNode**, **Instance**, **Mezzanine**, **Rack**, and **Switch**, compare **InternalGraphicType**:

- "(None)"
The graphic is no obstacle for the *Worker* who can walk through it or across it.
- "Bounding box"
The bounding box of the graphic is an obstacle for the *Worker* around which he has to walk.
- "Graphics"
The graphics of the graphics are an obstacle for the *Worker* around which he has to walk.

You can specify one of these settings for graphics of type **Box** and **Shape**:

- "(None)"
The box is no obstacle for the *Worker* who can walk through it or across it.
- "Bounding box"
The bounding box of the box is an obstacle for the *Worker* around which he has to walk.

Example

```
Station._3D.ObstacleForWorker := "Graphics"  
Station._3D.getGraphic("default", [2]).ObstacleForWorker := "(None)"
```

See also

[Worker Width \[model settings\]](#)/[Worker Width \[preferences\]](#)

[Edit 3D Properties \[dialog\]](#) > [Graphics \[tab\]](#) > [Obstacle for the Worker \[drop-down list\]](#)

[InternalGraphicType \[SimTalk\]](#)

[Show Obstacles](#)

[_3D.getGraphic \[SimTalk\]](#)

[_3D.ObstacleForWorker \[SimTalk\]](#) for objects

[Worker \[object\]](#)

optimizeByPruningTinyGraphics [SimTalk]

Optimizes the graphic designated by <Path> by pruning its tiny graphics, i.e., it removes all component graphics which individually fit within the cube designated by the parameter *BoundingBoxSideLength*.

Remarks

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

Type

Method

Syntax

```
<Path>.optimizeByPruningTinyGraphics(BoundingBoxSideLength: length)
```

Parameter

The parameter *BoundingBoxSideLength* is an array with a single value of data type *real*. It designates the size of the bounding box.

Example

```
.Models.Model._3D.getGraphic("deco",[1]).optimizeByPruningTinyGraphics([1],  
0.001)
```

See also

[Optimize Selected Graphic](#)

[Step 1: Prune Tiny Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

optimizeByStructureFlattening [SimTalk]

Optimizes the graphic designated by <Path>.

Remarks

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

Type

Method

Syntax

```
<Path>.optimizeByStructureFlattening(KeepGrouping:boolean,  
PreserveObstacles:boolean, RemovePolylinesAndPointSets:boolean[,  
PreserveReferences:boolean:=false])
```

Parameters

You can specify the following parameters:

- The parameter *KeepGrouping* of data type *boolean* sets if the structure of the object will be flattened while preserving its contents (*true*) or not (*false*).
- The parameter *PreserveObstacles* of data type *boolean* sets if obstacles are preserved (*true*) or not (*false*).
- The parameter *RemovePolylinesAndPointSets* of data type *boolean* sets if polylines and point sets are deleted (*true*) or not (*false*).
- The parameter *PreserveReferences* of data type *boolean* sets if multiple references to sub-graphics are going to be preserved and thus the model size is to be kept as small as possible (*true*) or not and thus allow the model size to grow by the optimization (*false*). If the parameter *KeepGrouping* is true as well, the parameter *PreserveReferences* has no effect.

Default Value of the Parameter

The default value is false.

Example

```
.Models.Model._3D.getGraphic("deco",  
[1]).optimizeByStructureFlattening(true, true, false)
```


See also

[Optimize Selected Graphic](#)

[Step 2: Flatten Structure](#)

[_3D.getGraphic \[SimTalk\]](#)

optimizeByVisibilityFilter [SimTalk]

Optimizes the graphic designated by <Path> i.e., it removes graphic components which are not visible from the outside.

Remarks

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

Type

Method

Syntax

```
<Path>.optimizeByVisibilityFilter(CullGranularity:integer,  
ReductionLevel:integer)
```

Parameters

You can specify the following parameters:

- The parameter *CullGranularity* of data type *integer* sets the granularity for deleting graphics. 0 is very fine granular, 3 is very coarse granular.
- The parameter *ReductionLevel* of data type *integer* sets how close the procedure looks at the graphic. 0 stands for a rather coarse look which causes a lot of graphics to be deleted very quickly, 3 stands for a very close look which takes a lot longer.

Examples

```
.Models.Model._3D.getGraphic("default", [1]).optimizeByVisibilityFilter(0,  
1)
```

```
.Models.Model._3D.getGraphic("deco", [2]).optimizeByVisibilityFilter(2, 1)
```

See also

Optimize Selected Graphic

Step 3: Visibility Filter

Position [SimTalk] - graphic

Sets the position of the graphic designated by <Path>.

Remarks

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.Position: length[3]
```

Assignment Value

You can assign an *array* of data type *length* with three values.

The values designate the position of the object on the **X**-axis, the **Y**-axis, and the **Z**-axis.

Examples

```
obj._3D.getGraphic("default", [1,7]).Position := [5,-5,0]
```

```
var p : length[3] := obj._3D.getGraphic("deco", [1,7]).Position
```

See also

[_3D.InheritGraphics \[SimTalk\]](#)

[_3D.Position \[SimTalk\]](#)

[X \[SimTalk\] - array](#)

[Y \[SimTalk\] - array](#)

[Z \[SimTalk\] - array](#)

[_3D.getGraphic \[SimTalk\]](#)

[Edit 3D Properties \[dialog\]](#) > [Transformation](#) > [Settings for the Position \[transformation\]](#)

removeTexture [SimTalk]

Deletes an existing texture from the graphic designated by <Path>.

Remarks

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

Type

Method

Syntax

```
<Path>.removeTexture
```

Example

```
MyFrame._3D.getGraphic("deco", [1,2]).removeTexture
```

See also

[Remove Texture \[3D graphic\]](#)

[_3D.getGraphic \[SimTalk\]](#)

Rotation [SimTalk] - graphic

Sets the rotation of the designated graphic designated by <Path>.

Remarks

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.Rotation:real/real[4]
```

Assignment Value

You can specify:

- A value of data type *real* which sets the rotation around the negative Z-axis at the position at which you inserted the object.
- An *array* of data type *real* with four values. The first value designates the angle, the remaining three values designate the components of the rotation axis at the position at which you inserted the object.

Return Value

The return value is an *array* of data type *real* with four values.

Examples

```
var a : any := MyStation._3D.getGraphic("default", [1]).Rotation
// returns an array of size 4
```

```
MyStation._3D.getGraphic("default", [1]).Rotation := 30
// rotates the graphic by 30 degrees
```

```
MyStation._3D.getGraphic("default", [1]).Rotation := [45, 0, 1, 0]
// rotates the graphic by 45 degrees around the Y-axis
```

See also

[Edit 3D Properties \[dialog\]](#) > [Transformation](#) > [Settings for the Rotation \[transformation\]](#)

[_3D.getGraphic \[SimTalk\]](#)

[_3D.Rotation \[SimTalk\]](#)

[F3DconcatenateRotations \[SimTalk\]](#)

Scale [SimTalk] - graphic

Sets the scaling of the graphic designated by <Path>.

Remarks

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.Scale:real/real[3]
```

Assignment Value

You can specify:

- A value of data type *real* which sets a uniform scaling.
- An *array* of data type *real* with three values. They designate the scaling factors along the X-axis, the Y-axis, and the Z-axis.

Examples

```
obj._3D.getGraphic("default", [1,7]).Scale := 1
```

```
var w : real[3] := obj._3D.getGraphic("deco", [1,7]).Scale
```

See also

[_3D.getGraphic \[SimTalk\]](#)

[_3D.Scale \[SimTalk\]](#)

[X \[SimTalk\] - array](#)

[Y \[SimTalk\] - array](#)

[Z \[SimTalk\] - array](#)

[Edit 3D Properties \[dialog\]](#) > [Transformation](#) > [Settings for the Scale \[transformation\]](#)

TransformationMatrix [SimTalk]

Sets the transformation matrix of the designated graphic group of the graphic designated by <Path>.

Remarks

Assigning the attribute deactivates graphic inheritance.

You cannot assign the attribute when the graphic is located in an automatically generated graphic group.

Type

Attribute

Syntax

```
<Path>.TransformationMatrix:real[4,4]/real:[16]
```

Assignment Value

To set the entire transformation matrix, you can:

- Specify a two-dimensional *array* of data type *real* with 4 times 4 values.

For the two-dimensional *array* the matrix looks like this:

m11	m12	m13	0
m21	m22	m23	0
m31	m32	m33	0
x	y	z	1

The sub-matrix **m** contains the rotation and the scaling. The values x, y, z designate the position.

- Specify a one-dimensional *array* of data type *real* with 16 values.

For the one-dimensional *array* the above values are arranged in the following sequence:

m11 m12 m13 0 m21 m22 m23 0 m31 m32 m33 0 x y z 1

Return Value

The return value is an *array* of data type *real* with four times four values.

Examples

```
MyStation._3D.getGraphic("default", [1]).TransformationMatrix := [1,0,0,0,  
0,1,0,0, 0,0,1,0, 3,4,0,1]
```

```
var m := MyStation._3D.getGraphic("default", [1]).TransformationMatrix  
m[2,4] += 3  
MyStation._3D.getGraphic("default", [2]).TransformationMatrix := m
```

See also

[Transformation](#)

[addGraphicTransformation \[SimTalk\]](#)

[_3D.TransformationMatrix \[SimTalk\]](#)